

Contents

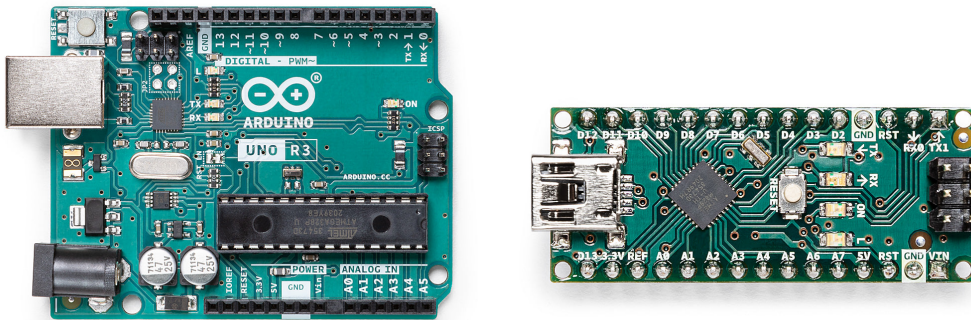
1	Hardware	3
1.1	Arduino	3
2	Coding your first few Sketches	5
2.1	Sketch Outline	5
2.2	Output	6
2.2.1	Blinking a Light Emitting Diode (LED)	6
2.2.2	Adding a counter	7
2.2.3	Using the Plotter	7
2.3	Input	9
2.3.1	Test for Echo	9
2.3.2	Calculator	9
2.3.3	Change Parameters	9
2.3.4	Sensors	9
3	Analog Sensing	11
3.1	Theory of Ohm's Law	11
3.2	Ohm's Law Applied	11
3.2.1	Potentimeters	11
3.2.2	Resistor Networks	11
3.2.3	Variable Restistance Sensors	11
4	Keeping Time	13

Chapter 1

Hardware

1.1 Arduino

The Arduino platform is a flexible, relatively easy to learn, microprocessor capable of sensing, communication, and control. There are many different versions of the Arduino, but we will focus on the two most common and fitting versions for this class. These are shown in Figure 1.1.



(a) Arduino Uno R3, the most popular Arduino hardware.

(b) Arduino Nano, a much smaller format.

Figure 1.1: Arduino Hardware, which is constantly evolving. These two versions are likely to continue to be available in some format in the near future.

You can think of the Arduino as a super simple computer, yet it's still different from the computer you probably use on a daily basis. The Arduino has a set of general purpose input and output pins (GPIO). This allows input and output of analog, digital, and communication protocols. Not all pins support all function though! See Figure 1.2 for details on the pins of the Uno. Don't worry about all of the acronyms in there yet though, as we'll go through those in the next few pages here.

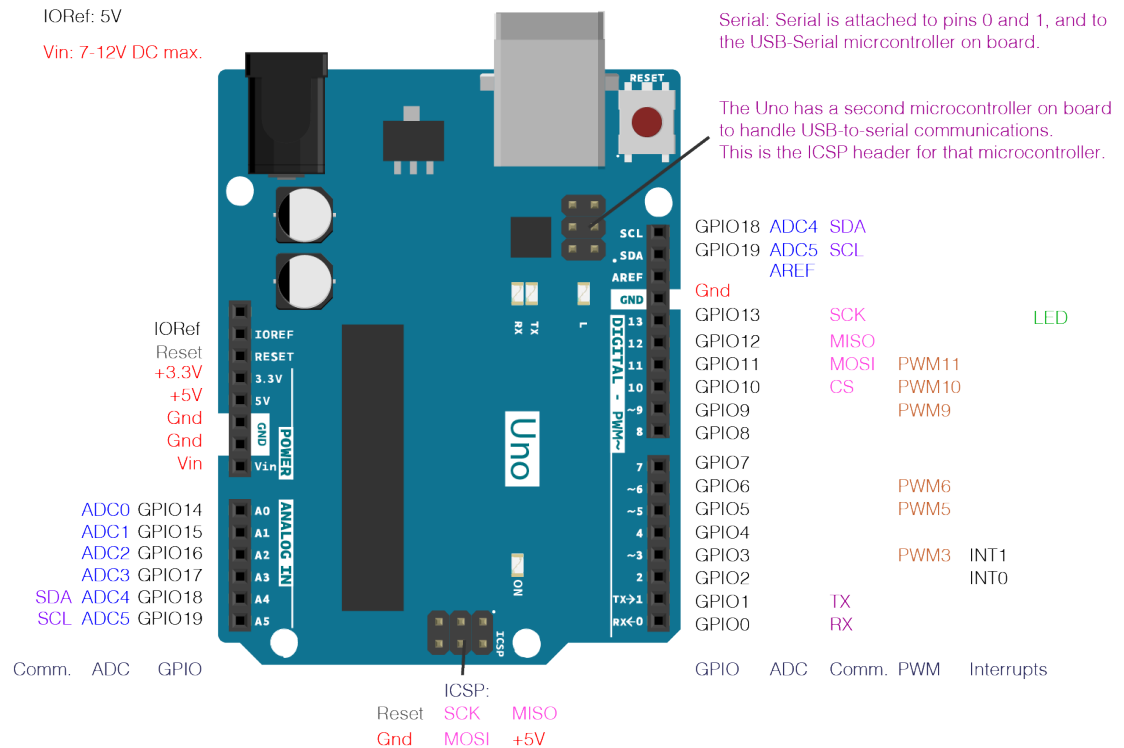


Figure 1.2: The Arduino Uno platform has 19 GPIO pins available for use. However, some of these pins support secondary functions. For example, you can see that A4 and A5 on the lower left are labeled as SDA ADC4 and SCL ADC5 respectively. SDA and SCL are communication type pins (discussed later). ADC are analog to digital converter pins (also discussed later). These pins cannot be used for both of these functions simultaneously.

Chapter 2

Coding your first few Sketches

2.1 Sketch Outline

We call Arduino programs sketches. There are three major parts to Arduino code layout. Let's start with the *second* section.

In this second section, as stated in the comments, code will run one time at start up. This is where we will put our code that defines how pins on the General Purpose Input/Output (GPIO) will behave. This could be setting a pin to be used as output, input, or even a communication interface.

In the *third* section we'll put the code that will run over and over again. This comes right after the `setup()` loop.

Finally, we need to declare variables that we want to be scoped globally. What does this mean? Any variables that we want to be able to access in the `setup()` and the `loop()`. If you're new to programming an Arduino, then it might be best to just assume that all variables should be global for now. There are indeed cases where you will want a variable to be accessible only inside of one of these, or a different function that you might write, but that's for later...

```
1 // put your variable declarations here.
2 // and your includes.
3
4 void setup() {
5     // put your setup code here, to run once:
6
7 }
8
9 void loop() {
10    // put your main code here, to run repeatedly:
11
12 }
```

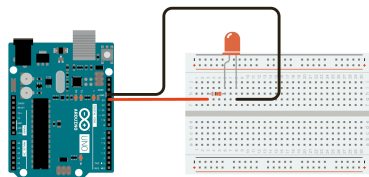
2.2 Output

2.2.1 Blinking a Light Emitting Diode (LED)

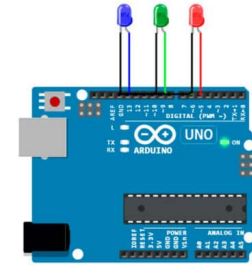
To start with, we'll get an LED connected to the Arduino to blink. Most Arduinos have a built in LED, but we're going to plug one in as a matter of practice. As shown here, we are using a breadboard and a resistor, in addition to the LED. However, in practice you can skip the breadboard and the resistor and connect your LED directly to the Arduino.

What about Ohm's Law?

We're going to make some assumptions for the moment, and will come back to this in 3.1.



(a) Following best practice, we should have a resistor in our circuit.



(b) But we can also generally get away with being a little lazy and skipping the resistor. More on this later!

Figure 2.1: Arduino LED examples.

Here's the code; let's look at what's really going on here.

```
1 void setup() {  
2   pinMode(13, OUTPUT); // Set pin 13 as an output  
3 }  
4  
5 void loop() {  
6   digitalWrite(13, HIGH); // Turn the LED on  
7   delay(1000); // Wait for one second  
8   digitalWrite(13, LOW); // Turn the LED off  
9   delay(1000); // Wait for one second  
10 }
```

First up, we are not using any variables to store state or values. We also are not using any special libraries, other supporting code, to do anything special. Thus, line 1 starts with our `setup()` call. Following this, we set the pin mode of GPIO pin 13 to be an output.

Next up, we have the `loop()`. These four lines turn the LED on by setting GPIO13 to “HIGH”, pause for 1 second (1000ms), turn the LED off by setting GPIO13 to “LOW”, pause for another 1 second, and then repeats. When we set a pin to “HIGH” we are setting it to “on”, or thinking in binary, “TRUE” at 5 volts (assuming we are using a 5 volt Arduino). Likewise, setting the pin to “LOW”, which you can also think of as “FALSE”, sets the pin to “off”.

Is there a state between “HIGH” and “LOW”?

Yes, but we need to save that for the section on pulse width modulation.

2.2.2 Adding a counter

In this course, we are using the Arduino as a platform for collecting data. Blinking an LED isn't really collecting data, or least not unless you want to sit by the LED and count how many times it blinks before the battery running the Arduino gives up. So let's introduce another function that is built into the Arduino, one that will let us communicate with the Arduino.

For now, we have been plugging our Arduino into our computer to upload our programs, and incidentally, to power the Arduino. With the introduction of this next function, we are assuming that is still the case.

```
1 int ledBlinks = 0;      // Create a variable to store the ←
   number of times the LED has blinked for us
2
3 void setup() {
4   pinMode(13, OUTPUT); // Set pin 13 as an output
5   Serial.begin(9600);  // Define that we will be using the ←
   serial port, and 9600 baud
6 }
7
8 void loop() {
9   digitalWrite(13, HIGH); // Turn the LED on
10  delay(1000); // Wait for one second
11  digitalWrite(13, LOW); // Turn the LED off
12  delay(1000); // Wait for one second
13  ledBlinks = ledBlinks + 1;
14  Serial.print(ledBlinks);
15 }
```

What is baud?

Baud, or baud rate, is how fast we will communicate between devices. This is bits per second.

2.2.3 Using the Plotter

The Arduino IDE has a built in data plotter. This is especially helpful as we think about reading sensor data in realtime and debugging. Let's modify our counter sketch to instead count up to some number, then count back down to another, then up again, and so on. We are going to make a saw tooth function.

Saw Tooth Algorithm

```
'x = 0
x_max = 20
x_min = 0
direction = 1
while True
    if x >= x_max
        direction = -1
    else if x <= x_min
        direction = 1
    end if
    x = x + direction
    print x
end while
```

This saw tooth algorithm should oscillate x between 0 and 20. To translate this algorithm into Arduino code, we need to establish our variables (lines 1 so 5), setup the serial communication (lines), and then loop through evaluating the state of x while making changes to the direction (lines) and thus the value of x as appropriate (lines).

Ints

Arduino Ints are made up of two bytes and can range from -32,768 to 32,767.

```
1 int x = 0;          // to store the value we want to be outputting
2 int x_max = 20;     // we can make this a larger number, try it ←
   after trying 20 first
3 int x_min = 0;      // likewise to x_max, this can be changed ←
   after you give 0 a shot
4 int direction = 1;  // see the note on ints
5
6 void setup() {
7     // we don't really need the LED in this example, though you ←
   could use two LEDs to indicate the direction
8     Serial.begin(9600); // 9600 baud
9 }
10
11 void loop() {
12     if x >= x_max {
13         direction = -1;
14     } else {
15         direction = 1;
16     }
17     x = x + direction;
18     Serial.print(x);
19     delay(500); // Wait for half a second
20 }
```

Then show an image of the plotter in action.

2.3 Input

2.3.1 Test for Echo

take some input, and echo that back

2.3.2 Calculator

take two single digit inputs, and do addition

2.3.3 Change Parameters

take input and make things behave differently

restart and the defaults

2.3.4 Sensors

maybe start with a digital sensor, like an accelerometer?

then introduce analog in the next chapter?

Chapter 3

Analog Sensing

Most all sensors are, at heart, analog in some way. Many of them, such as the accelerometer we used in the previous chapter, have an analog-to-digital converter built in. But not all do. Before we can discuss analog sensors in great detail, we need to cover some basic electrical engineering basics.

3.1 Theory of Ohm's Law

overview of ohms law

3.2 Ohm's Law Applied

then into resistor potentiometers, and resistor networks

3.2.1 Potentiometers

3.2.2 Resistor Networks

with how to read resistors

3.2.3 Variable Resistance Sensors

such as flex sensors, force sensors, light sensors, etc

Chapter 4

Keeping Time

Chapter 5

Saving the Data